Logical Database Design

Objectives

- Translating the E-R model to A logical data Model
- Validating the Logical Data model with Normalization
- The purpose of normalization.
- How normalization can be used when designing a relational database.
- The potential problems associated with redundant data in base relations.
- The concept of functional dependency, which describes the relationship between attributes.
- The characteristics of functional dependencies used in normalization.

Objectives Cont'd...

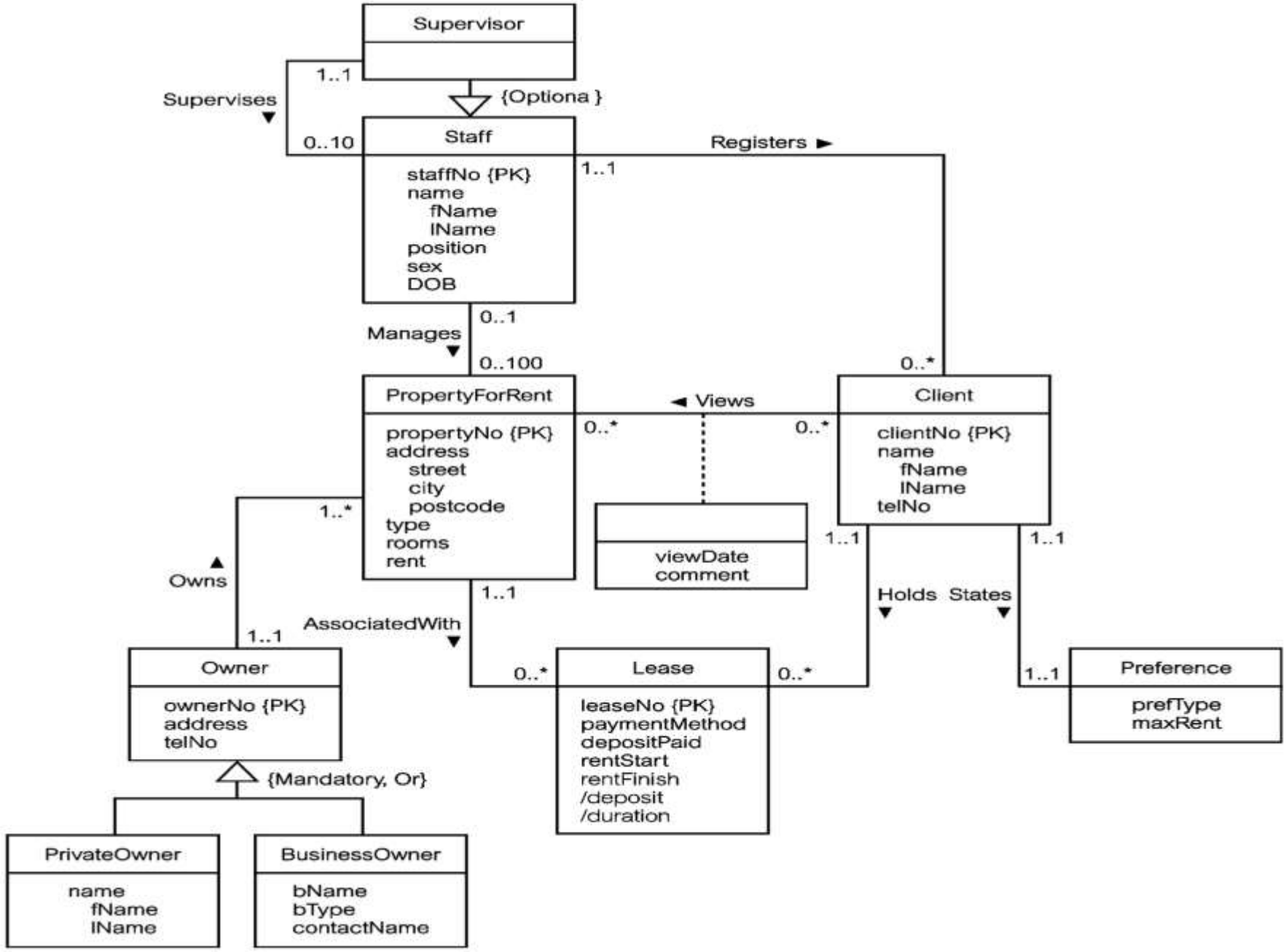
- How to undertake the process of normalization.
- How normalization uses functional dependencies to group attributes into relations that are in a known normal form.
- How to identify the most commonly used normal forms, namely First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).
- The problems associated with relations that break the rules of 1NF, 2NF, or 3NF.

Build and Validate Logical Data Model

- **To translate the conceptual data model (E-R) into a logical data model and then to validate this model to check that it is structurally correct using normalization and supports the required business transactions.**

Build and Validate Logical Data Model

- **Derive relations for logical data model**
  - **To create relations for the logical data model to represent the entities, relationships, and attributes that have been identified.**
- **Three basic Rules**
  - **Mapping Entities:** Entity names **will be automatically be Table names**
  - **Mapping of attributes:** attributes will be columns of the respective tables.
    - **Atomic or single-valued or derived or stored attributes will be columns**
    - **Composite attributes: the parent attribute will be ignored and the decomposed attributes (child attributes) will be columns of the table.**
    - **Multi-valued attributes: will be mapped to a new table where the primary key of the main table will be posted for cross referencing**
  - **Relationships: according to their cardinality type**


Supervisor
1..1
Supervises
{Optiona }
0..10
Staff
staffNo {PK}
name
fName
lName
position
sex
DOB
Registers
1..1
0..1
Manages
0..100
0..*
PropertyForRent
propertyNo {PK}
address
street
city
postcode
type
rooms
rent
Views
0..*
0..*
Client
clientNo {PK}
name
fName
lName
telNo
viewDate
comment
1..*
Owns
1..1
1..1
AssociatedWith
1..1
1..1
Holds
States
1..1
Owner
ownerNo {PK}
address
telNo
0..*
Lease
leaseNo {PK}
paymentMethod
depositPaid
rentStart
rentFinish
/deposit
/duration
0..*
Preference
prefType
maxRent
{Mandatory, Or}
PrivateOwner
name
fName
lName
BusinessOwner
bName
bType
contactName

Derive relations for logical data model

Weak entity types

- For each weak entity in the data model, The **primary key of** a weak entity is partially or fully derived from each owner entity and so the identification of the primary key of a weak entity cannot be made until after all the relationships with the owner entities have been mapped.

Derive relations for logical data model

- **One-to-one (1:1) recursive relationships** Unary
  - For a 1:1 recursive relationship, follow the rules for participation as described above for a 1:1 relationship.
    - mandatory participation on both sides, represent the recursive relationship as a single relation with two copies of the primary key.
    - mandatory participation on only one side, option to create a single relation with two copies of the primary key, or to create a new relation to represent the relationship. The new relation would only have two attributes, both copies of the primary key. As before, the copies of the primary keys act as foreign keys and have to be renamed to indicate the purpose of each in the relation.
    - optional participation on both sides, again create a new relation as described above.

Derive relations for logical data model

- **One-to-one (1:1) binary relationship types**
  - Creating relations to represent a 1:1 relationship is more complex as the cardinality cannot be used to identify the parent and child entities in a relationship. Instead, the participation constraints are used to decide whether it is best to represent the relationship by combining the entities involved into one relation or by creating two relations and posting a copy of the primary key from one relation to the other.
  - Consider the following
    - *(a) mandatory* participation on *both* sides of 1:1 relationship;
    - *(b) mandatory* participation on *one* side of 1:1 relationship;
    - *(c) optional* participation on *both* sides of 1:1 relationship.

Derive relations for logical data model

- **(a) *Mandatory* participation on *both* sides of 1:1 relationship**
  - Combine entities involved into one relation and choose one of the primary keys of original entities to be primary key of the new relation, while the other (if one exists) is used as an alternate key.

- **(b) *Mandatory* participation on *one* side of a 1:1 relationship**
  - Identify parent and child entities using participation constraints. Entity with optional participation in relationship is designated as parent entity, and entity with mandatory participation is designated as child entity. A copy of primary key of the parent entity is placed in the relation representing the child entity. If the relationship has one or more attributes, these attributes should follow the posting of the primary key to the child relation.

Derive relations for logical data model

- (c) ***Optional*** **participation on *both* sides of a 1:1 relationship**
  - In this case, the designation of the parent and child entities is arbitrary unless we can find out more about the relationship that can help a decision to be made one way or the other.

Derive relations for logical data model

- **One-to-many (1:*) binary relationship types**
  - **For each 1:* binary relationship, the entity on the 'one side' of the relationship is designated as the parent entity and the entity on the 'many side' is designated as the child entity. To represent this relationship, post a copy of the primary key attribute(s) of parent entity into the relation representing the child entity, to act as a foreign key.**
  - **Attributes of the relationship(if any) are posted on the many side**

Derive relations for logical data model

- **Many-to-many (*:*) binary relationship types**
  - **Create a relation to represent the relationship and include any attributes that are part of the relationship. We post a copy of the primary key attribute(s) of the entities that participate in the relationship into the new relation, to act as foreign keys. These foreign keys will also form the primary key of the new relation, possibly in combination with some of the attributes of the relationship.**
- **Same thing holds for an associative entity.**

Derive relations for logical data model

- **Complex relationship types**
  - Create a relation to represent the relationship and include any attributes that are part of the relationship. Post a copy of the primary key attribute(s) of the entities that participate in the complex relationship into the new relation, to act as foreign keys. Any foreign keys that represent a 'many' relationship (for example, 1..*, 0..*) generally will also form the primary key of this new relation, possibly in combination with some of the attributes of the relationship.

Derive relations for logical data model

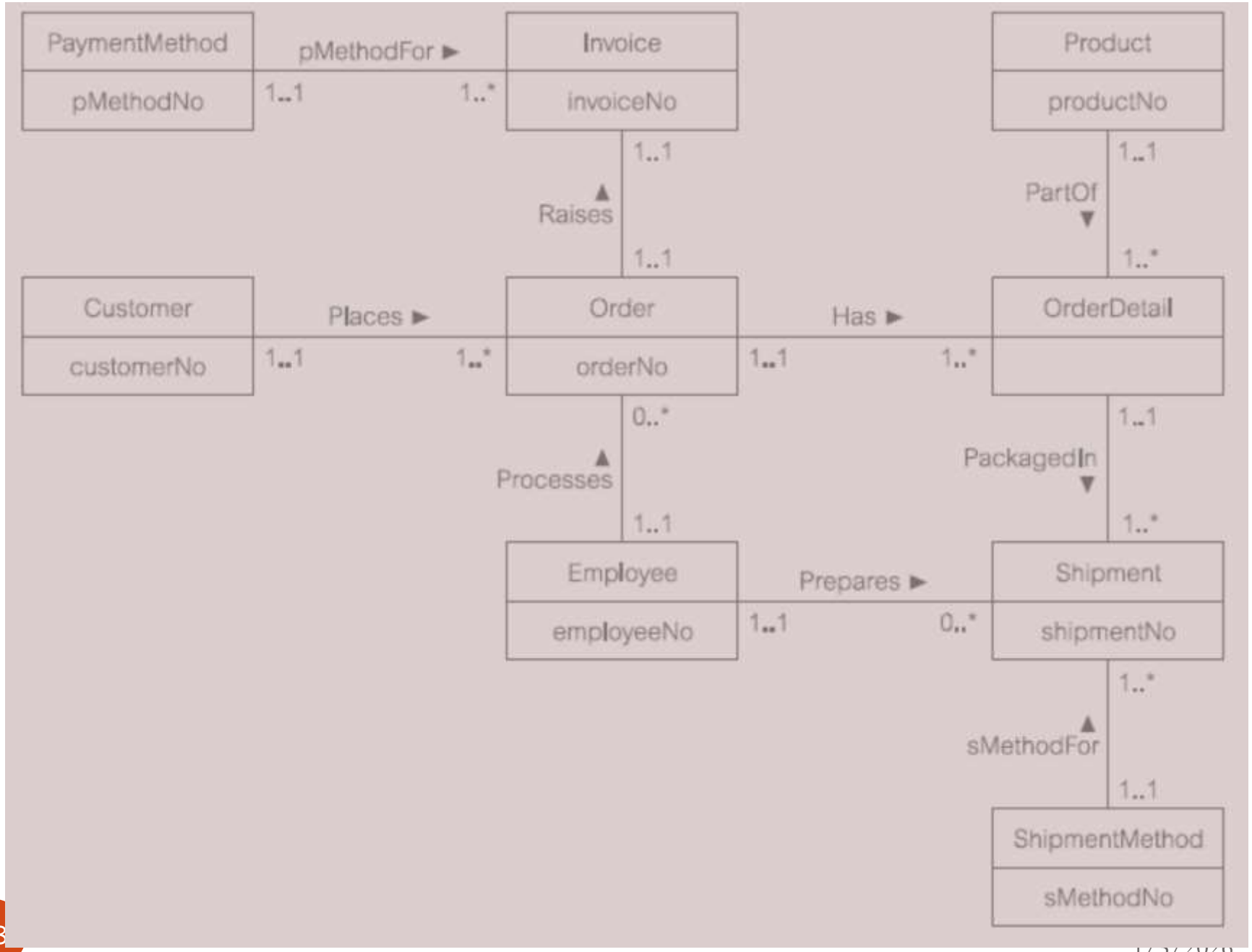
- **Superclass/subclass relationship types**
  - **Identify superclass entity as parent entity and subclass entity as the child entity. There are various options on how to represent such a relationship as one or more relations.**
  - **The selection of the most appropriate option is dependent on a number of factors such as the disjointness and participation constraints on the superclass/subclass relationship, whether the subclasses are involved in distinct relationships, and the number of participants in the superclass/subclass relationship.**

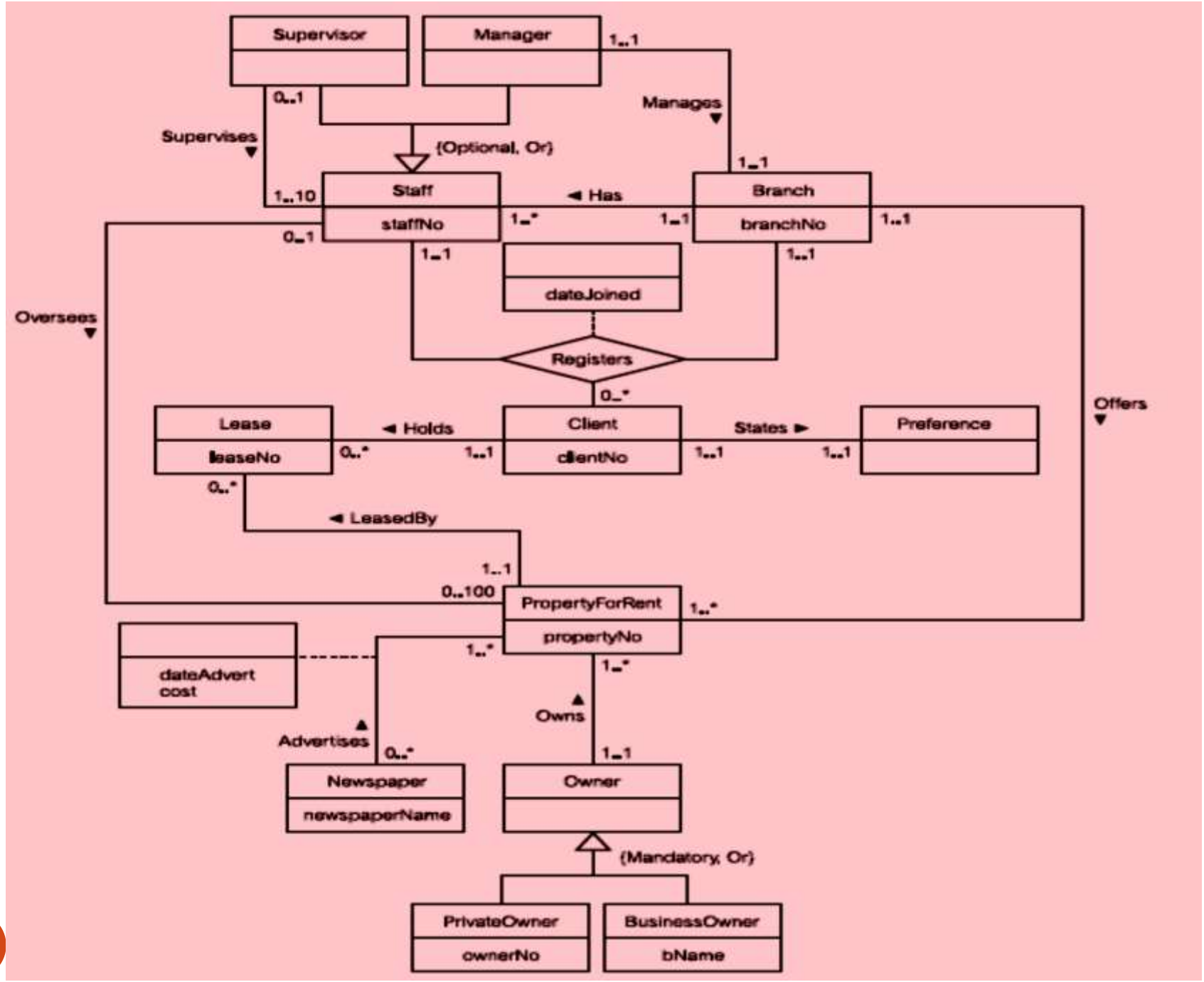
Guidelines for representation of superclass / subclass relationship

Participation constraint	Disjoint constraint	Relations required
Mandatory	Nondisjoint {And}	Single relation (with one or more discriminators to distinguish the type of each tuple)
Optional	Nondisjoint {And}	Two relations: one relation for superclass and one relation for all subclasses (with one or more discriminators to distinguish the type of each tuple)
Mandatory	Disjoint {Or}	Many relations: one relation for each combined superclass/subclass
Optional	Disjoint {Or}	Many relations: one relation for superclass and one for each subclass

Summary of how to map entities and relationships to relations

Entity/Relationship	Mapping
Strong entity	Create relation that includes all simple attributes.
Weak entity	Create relation that includes all simple attributes (primary key still has to be identified after the relationship with each owner entity has been mapped).
1:* binary relationship	Post primary key of entity on 'one' side to act as foreign key in relation representing entity on 'many' side. Any attributes of relationship are also posted to 'many' side.
1:1 binary relationship:	
(a) Mandatory participation on both sides	Combine entities into one relation.
(b) Mandatory participation on one side	Post primary key of entity on 'optional' side to act as foreign key in relation representing entity on 'mandatory' side.
(c) Optional participation on both sides	Arbitrary without further information.
Superclass/subclass relationship	See Table 16.1.
: binary relationship, complex relationship	Create a relation to represent the relationship and include any attributes of the relationship. Post a copy of the primary keys from each of the owner entities into the new relation to act as foreign keys.
Multi-valued attribute	Create a relation to represent the multi-valued attribute and post a copy of the primary key of the owner entity into the new relation to act as a foreign key.


PaymentMethod
pMethodNo
pMethodFor ►
1..1
1..*
Invoice
invoiceNo
Product
productNo
1..1
Raises
1..1
PartOf
1..1
1..*
Customer
customerNo
Places ►
1..1
1..*
Order
orderNo
Has ►
1..1
1..*
OrderDetail
0..*
Processes
1..1
1..1
PackagedIn
1..*
Employee
employeeNo
Prepares ►
1..1
0..*
Shipment
shipmentNo
1..*
sMethodFor
1..1
ShipmentMethod
sMethodNo


Supervisor
Manager
1..1
0..1
Manages
Supervises
{Optional, Or}
1..1
1..10
Staff
staffNo
Has
Branch
branchNo
1..*
1..1
1..1
0..1
1..1
1..1
dateJoined
Oversees
Registers
0..*
Offers
Lease
leaseNo
Holds
Client
clientNo
States
Preference
0..*
1..1
1..1
1..1
0..*
LeasedBy
1..1
0..100
PropertyForRent
propertyNo
1..*
dateAdvert
cost
1..*
1..*
Owns
Advertises
0..*
1..1
Newspaper
newspaperName
Owner
{Mandatory, Or}
PrivateOwner
ownerNo
BusinessOwner
bName

Purpose of Normalization

- **Normalization is a technique for producing a set of suitable relations that support the data requirements of an enterprise.**

- Is used to the design **"What"** aspect of the database in terms of an implementation data model

Purpose of Normalization

- **Characteristics of a suitable set of relations include:**
  - **the *minimal* number of attributes necessary to support the data requirements of the enterprise;**
  - **attributes with a close logical relationship are found in the same relation;**
  - ***minimal* redundancy with each attribute represented only once with the important exception of attributes that form all or part of foreign keys.**

Purpose of Normalization

- **The benefits of using a database that has a suitable set of relations is that the database will be:**
  - **easier for the user to access and maintain the data;**
  - **take up minimal storage space on the computer.**

How Normalization Supports Database Design

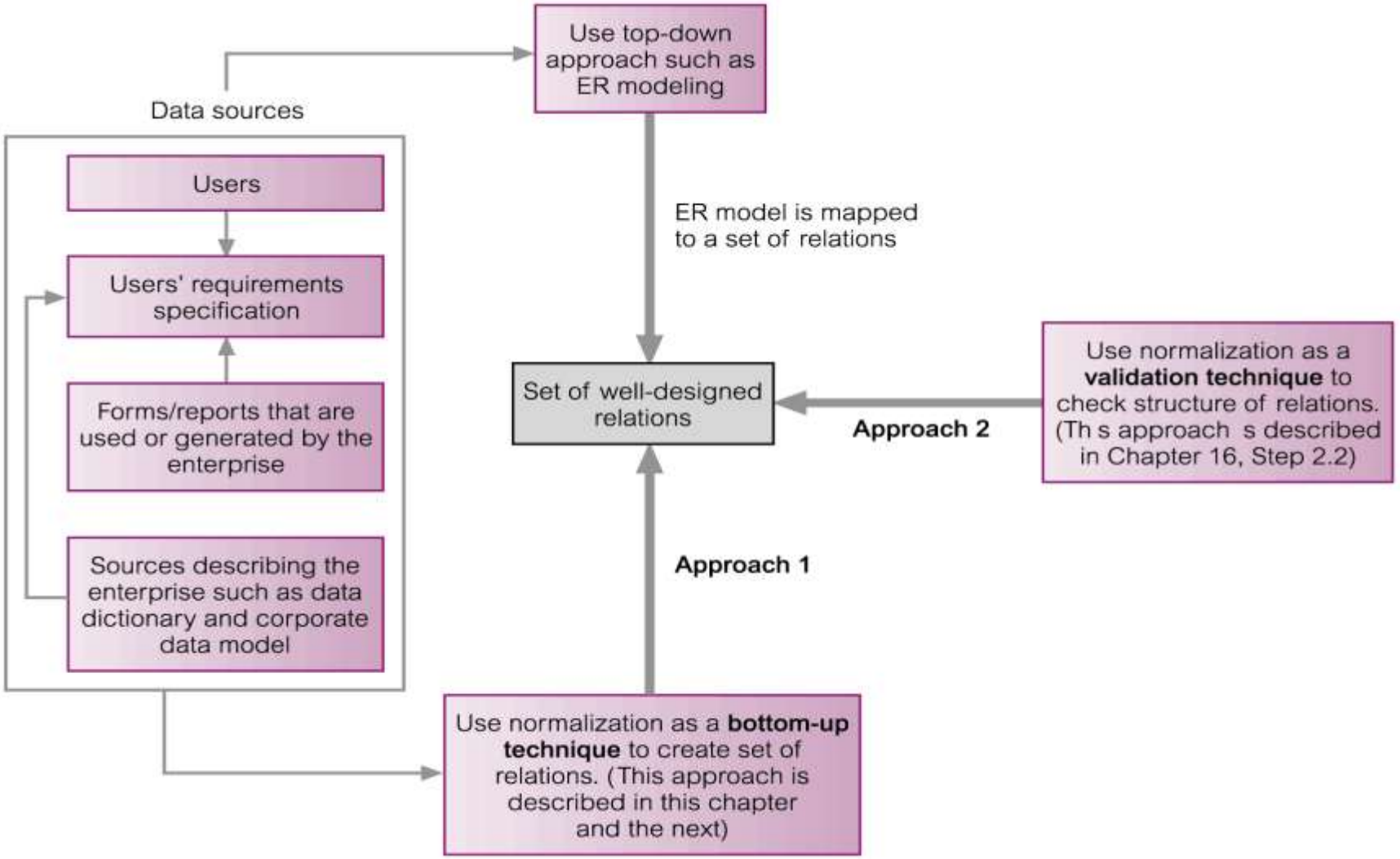


Figure 13.1 How normalization can be used to support database design.

Data Redundancy and Update Anomalies

- **Major aim of relational database design is to group attributes into relations to minimize data redundancy.**

Data Redundancy and Update Anomalies

- **Potential benefits for implemented database include:**
  - **Updates to the data stored in the database are achieved with a minimal number of operations thus reducing the opportunities for data inconsistencies.**
  - **Reduction in the file storage space required by the base relations thus minimizing costs.**

Data Redundancy and Update Anomalies

- **Problems associated with data redundancy are illustrated by comparing the Staff and Branch relations with the StaffBranch relation.**

Data Redundancy and Update Anomalies

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Staff Branch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Data Redundancy and Update Anomalies

- **StaffBranch relation has redundant data; the details of a branch are repeated for every member of staff.**

- **In contrast, the branch information appears only once for each branch in the Branch relation and only the branch number (branchNo) is repeated in the Staff relation, to represent where each member of staff is located.**

Data Redundancy and Update Anomalies

- **Relations that contain redundant information may potentially suffer from update anomalies.**

- **Types of update anomalies include**
  - **Insertion:- impossibility, Omission +Inconsistency**
  - **Deletion:- Loss of dependent data, Omission**
  - **Modification:- Inconsistency, Omission**
- **<u>So what does normalization do?</u>**
  - **Decompose relations to smaller/non-redundant but efficient storage & processing**

Lossless-join and Dependency Preservation Properties

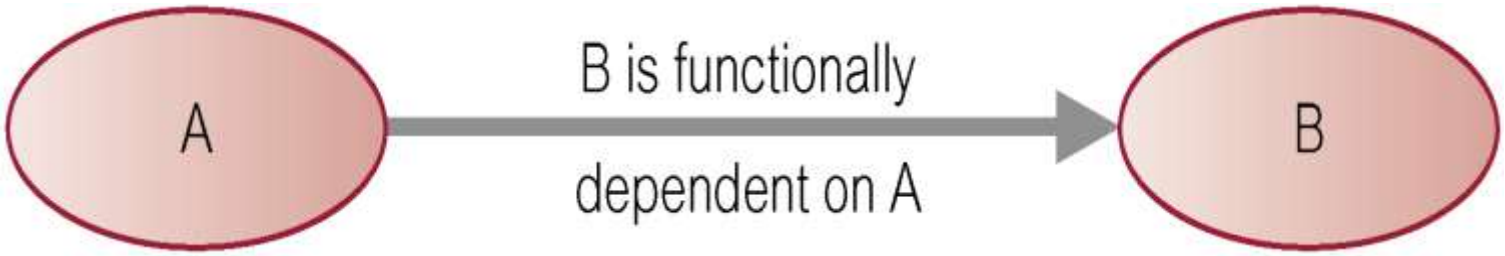
- **Two important properties of decomposition.**
  - ***Lossless-join property* enables us to find any instance of the original relation from corresponding instances in the smaller relations.**
  - ***Dependency preservation property* enables us to enforce a constraint on the original relation by enforcing some constraint on each of the smaller relations.**

Functional Dependencies

- **Important concept associated with normalization.**
- **Functional dependency describes relationship between attributes.**
- **For example, if A and B are attributes of relation R, B is functionally dependent on A (denoted A → B), if each value of A in R is associated with exactly one value of B in R.**

Characteristics of Functional Dependencies

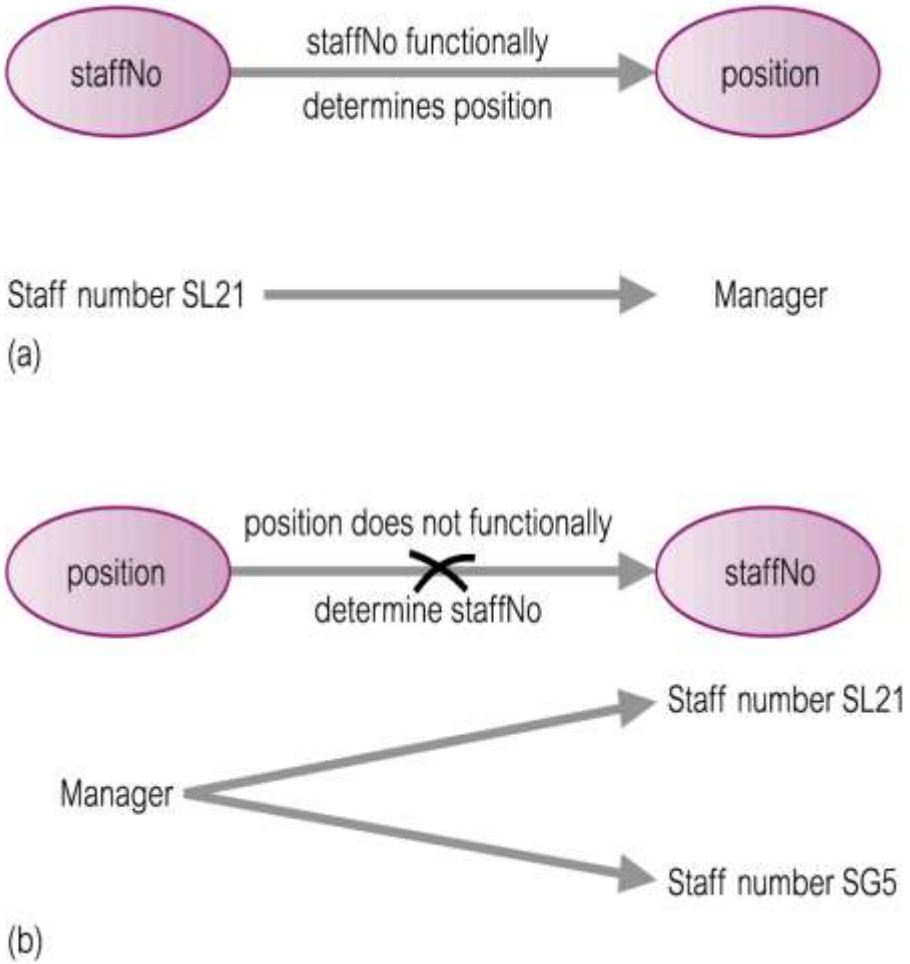
- **Property of the meaning or semantics of the attributes in a relation.**
- **Diagrammatic representation.**



- **The *determinant* of a functional dependency refers to the attribute or group of attributes on the left-hand side of the arrow.**

An Example Functional Dependency

Figure 13.5
(a) staffNo functionally determines position (staffNo → position); (b) position does *not* functionally determine staffNo (position ↛ staffNo).



Example Functional Dependency that holds for all Time

- **Consider the values shown in staffNo and sName attributes of the Staff relation (see Slide 27).**

- **Based on sample data, the following functional dependencies appear to hold.**

staffNo → sName

sName → staffNo

Example Functional Dependency that holds for all Time

- **However, the only functional dependency that remains true for all possible values for the staffNo and sName attributes of the Staff relation is:**

$$\text{staffNo} \rightarrow \text{sName}$$

Characteristics of Functional Dependencies

- **Determinants should have the minimal number of attributes necessary to maintain the functional dependency with the attribute(s) on the right hand-side.**

- **This requirement is called *full functional dependency*.**

Characteristics of Functional Dependencies

- **Full functional dependency indicates that if A and B are attributes of a relation, B is fully functionally dependent on A, if B is functionally dependent on A, but not on any proper subset of A.**

Example Full Functional Dependency

- **Exists in the Staff relation (see Slide 27).**

  staffNo, sName → branchNo

- **True - each value of (staffNo, sName) is associated with a single value of branchNo.**

- **However, branchNo is also functionally dependent on a subset of (staffNo, sName), namely staffNo. Example above is a *partial dependency*.**

Characteristics of Functional Dependencies

- **Main characteristics of functional dependencies used in normalization:**
  - **There is a *one-to-one* relationship between the attribute(s) on the left-hand side (determinant) and those on the right-hand side of a functional dependency.**
  - **Holds for *all* time.**
  - **The determinant has the *minimal* number of attributes necessary to maintain the dependency with the attribute(s) on the right hand-side.**

Transitive Dependencies

- **Important to recognize a transitive dependency because its existence in a relation can potentially cause update** anomalies.

- **Transitive dependency describes a condition where A, B, and C are attributes of a relation such that if A → B and B → C, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).**

Example Transitive Dependency

- **Consider functional dependencies in the StaffBranch relation (see Slide 27).**

staffNo → sName, position, salary, branchNo, bAddress

branchNo → bAddress

- **Transitive dependency, branchNo → bAddress exists on staffNo via branchNo.**

The Process of Normalization

- **Formal technique for analyzing a relation based on its primary key and the functional dependencies between the attributes of that relation.**

- **Often executed as a series of steps. Each step corresponds to a specific normal form, which has known properties.**

Identifying Functional Dependencies

- **Identifying all functional dependencies between a set of attributes is relatively simple if the meaning of each attribute and the relationships between the attributes are well understood.**

- **This information should be provided by the enterprise in the form of discussions with users and/or documentation such as the users' requirements specification.**

Identifying Functional Dependencies

- **However, if the users are unavailable for consultation and/or the documentation is incomplete then depending on the database application it may be necessary for the database designer to use their common sense and/or experience to provide the missing information.**

Example - Identifying a set of functional dependencies for the StaffBranch relation

- **Examine semantics of attributes in StaffBranch relation (see Slide 27). Assume that position held and branch determine a member of staff's salary.**

Example - Identifying a set of functional dependencies for the StaffBranch relation

- **With sufficient information available, identify the functional dependencies for the StaffBranch relation as:**

staffNo → sName, position, salary, branchNo, bAddress

branchNo → bAddress

bAddress → branchNo

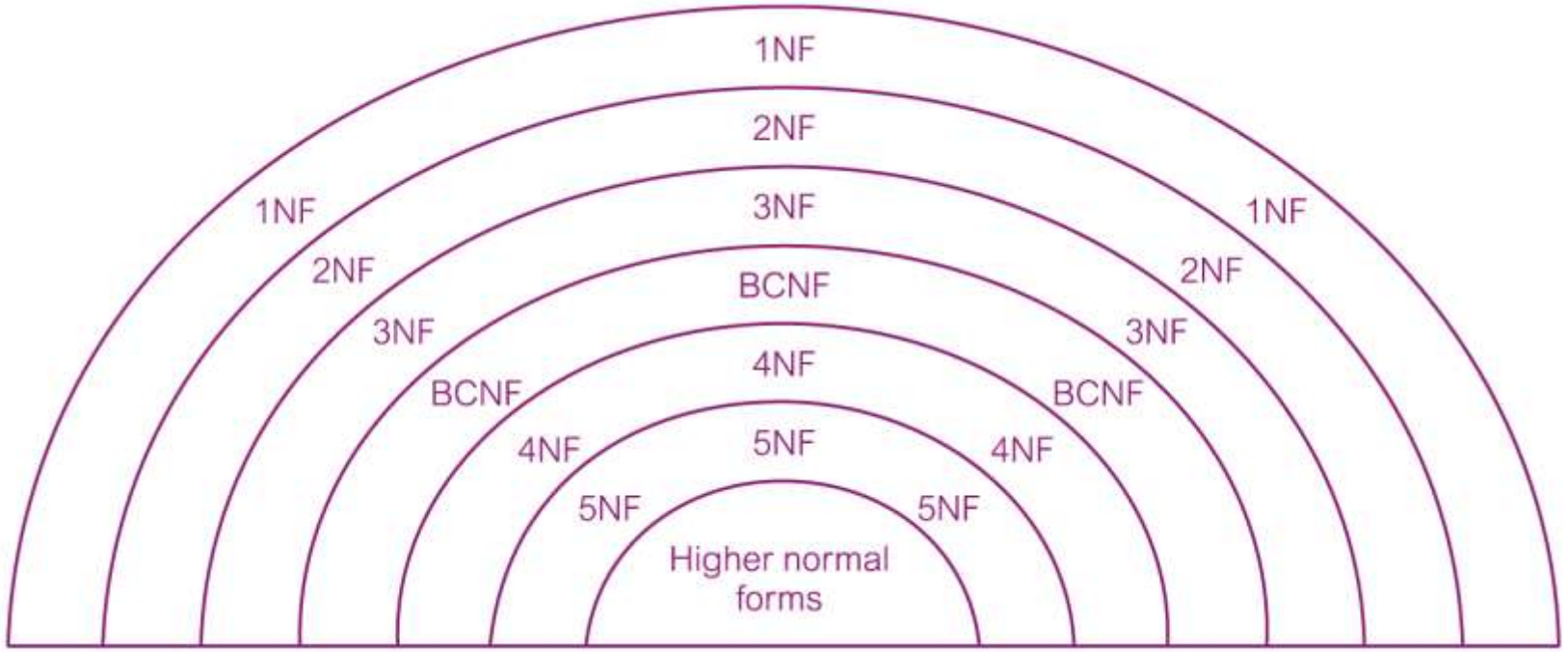
branchNo, position → salary

bAddress, position → salary

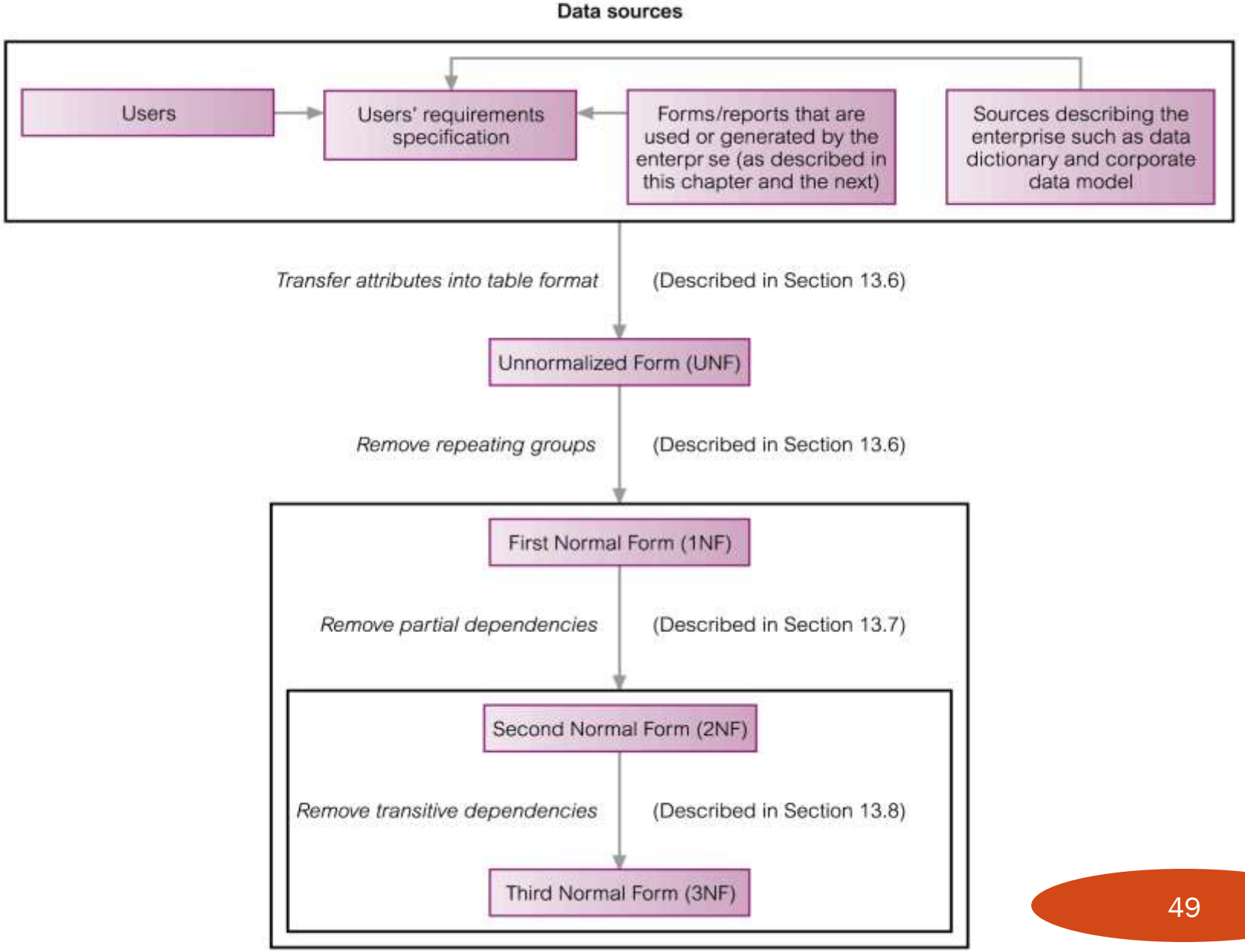
The Process of Normalization

- **As normalization proceeds, the relations become progressively more restricted (stronger) in format and also less vulnerable to update anomalies.**

The Process of Normalization



The Process of Normalization



Unnormalized Form (UNF)

- **A table that contains one or more repeating groups.**

- **To create an unnormalized table**
  - **Transform the data from the information source (e.g. form) into table format with columns and rows.**

First Normal Form (1NF)

- **A relation in which the intersection of each row and column contains one and only one value.**

UNF to 1NF

- **Nominate an attribute or group of attributes to act as the key for the unnormalized table.**

- **Identify the repeating group(s) in the unnormalized table which repeats for the key attribute(s).**

UNF to 1NF

- **Remove the repeating group by**
  - **Entering appropriate data into the empty columns of rows containing the repeating data ('flattening' the table).**

  Or by

  - **Placing the repeating data along with a copy of the original key attribute(s) into a separate relation.**

Second Normal Form (2NF)

- **Based on the concept of full functional dependency.**

- **Full functional dependency indicates that if**
  - **A and B are attributes of a relation,**
  - **B is fully dependent on A if B is functionally dependent on A but not on any proper subset of A.**

Second Normal Form (2NF)

- **A relation that is in 1NF and every non-primary-key attribute is fully functionally dependent on the primary key.**

1NF to 2NF

- **Identify the primary key for the 1NF relation.**
- **Identify the functional dependencies in the relation.**
- **If partial dependencies exist on the primary key remove them by placing them in a new relation along with a copy of their determinant.**

Third Normal Form (3NF)

- **Based on the concept of transitive dependency.**

- **Transitive Dependency is a condition where**
  - **A, B and C are attributes of a relation such that if A → B and B → C,**
  - **then C is transitively dependent on A through B. (Provided that A is not functionally dependent on B or C).**
    - **i.e. B\→A Nor C\→A**

Third Normal Form (3NF)

- **A relation that is in 1NF and 2NF and in which no non-primary-key attribute is transitively dependent on the primary key.**

2NF to 3NF

- **Identify the primary key in the 2NF relation.**
- **Identify functional dependencies in the relation.**
- **If transitive dependencies exist on the primary key remove them by placing them in a new relation along with a copy of their determinant.**

General Definitions of 2NF and 3NF

- **Second normal form (2NF)**
  - **A relation that is in first normal form and every non-primary-key attribute is fully functionally dependent on *any candidate key*.**

- **Third normal form (3NF)**
  - **A relation that is in first and second normal form and in which no non-primary-key attribute is transitively dependent on *any candidate key*.**

Boyce–Codd Normal Form (BCNF)

- **Based on functional dependencies that take into account all candidate keys in a relation, however BCNF also has additional constraints compared with the general definition of 3NF.**

- **Boyce–Codd normal form (BCNF)**
  - **A relation is in BCNF if and only if every determinant is a candidate key.**

Boyce–Codd Normal Form (BCNF)

- **Difference between 3NF and BCNF is that for a functional dependency A → B, 3NF allows this dependency in a relation if B is a primary-key attribute and A is not a candidate key. Whereas, BCNF insists that for this dependency to remain in a relation, A must be a candidate key.**

- **Every relation in BCNF is also in 3NF. However, a relation in 3NF is not necessarily in BCNF.**

Boyce–Codd Normal Form (BCNF)

- **Violation of BCNF is quite rare.**
- **The potential to violate BCNF may occur in a relation that:**
  - **contains two (or more) composite candidate keys;**
  - **the candidate keys overlap, i.e. have at least one attribute in common.**